Apuntes tema 2

Computación paralela

¿Qué es?

Es un tipo de procesamiento de información en el que **varias operaciones o cálculos se realizan simultáneamente.**

¿Cuál es su objetivo?

Mejorar el rendimiento y la eficiencia al dividir una tarea grande en **tareas más pequeñas que pueden ejecutarse simultáneamente** acortando el tiempo de procesamiento total.

Es útil para abordar problemas que requieren una gran cantidad de cálculos como la renderización de gráficos en aplicaciones multimedia.

¿Por qué es útil para el Big Data?

La computación paralela es crucial para el Big Data porque:

1. **Manejo Eficiente:** Permite dividir grandes conjuntos de datos en partes más pequeñas para un procesamiento más rápido.
2. **Rendimiento Mejorado: U**tiliza múltiples procesadores para ejecutar tareas simultáneamente, mejorando el rendimiento.
3. **Paralelización de Tareas:** Distribuye tareas independientes entre varios procesadores para acelerar operaciones complejas.
4. **Escalabilidad:** Es escalable, permitiendo agregar más recursos de procesamiento para manejar conjuntos de datos en crecimiento.
5. **Algoritmos Paralelizables:** Aprovecha algoritmos y técnicas paralelizables para un procesamiento eficiente.
6. **Distribución de Cargas de Trabajo:** Facilita la distribución de cargas de trabajo en sistemas distribuidos para un procesamiento a gran escala.

Ejemplo de utilización del paralelismo en Big Data

El principal ejemplo es el MapReduce que funciona de la siguiente manera:

1. Tenemos un conjunto de datos que contiene información sobre clics de usuarios en un sitio web y quieres calcular el número total de clics por usuario.
2. **Fase de Map:** Se distribuye el conjunto de datos a través de un clúster de computadoras. Cada máquina (o nodo) ejecuta una función de "map" para procesar solo una porción de los datos y emite pares clave-valor que representan información relevante.
3. **Shuffle y Sort:** Los resultados de la fase de "map" se agrupan y se ordenan por clave a través de una operación llamada "shuffle and sort".
4. **Fase de Reduce:** Se realiza una función de reducción que toma los pares clave-valor ordenados, los procesa y produce el resultado final, en este caso, el total de clics por usuario.

Computación distribuida

¿Qué es?

Es un modelo de procesamiento de información en el que **varias computadoras o nodos colaboran entre sí para realizar una tarea o resolver un problema.**

¿Cuál es su objetivo?

Un sistema distribuido tiene varios objetivos:

1. **Cooperación:** Los nodos trabajan de manera cooperativa para lograr un objetivo común.
2. **Interconexión:** Los nodos están conectados entre sí a través de una red. Esta interconexión puede ser: Redes locales (LAN) o Redes globales (Internet).
3. **Descentralización:** La toma de decisiones y el procesamiento se distribuyen entre los nodos.
4. **Escalabilidad:** Pueden crecer agregando más nodos a medida que aumenta la carga de trabajo o la complejidad de la tarea.
5. **Tolerancia a Fallos:** Si un nodo falla, otros pueden continuar trabajando, evitando que un solo punto de fallo afecte todo el sistema.

6. **Eficiencia y Rendimiento:** La distribución de tareas entre múltiples nodos lleva a un mejor rendimiento y eficiencia.

¿Por qué es útil para el Big Data?

Para resolver problemas. El ejemplo de MapReduce es una combinación de computación paralela y distribuida, concretamente:

1. Resolver un problema en diferentes tareas.
2. Diferentes tareas que se llevan a cabo en diferentes nodos parte de un sistema distribuido.

Trabajando con Hadoop

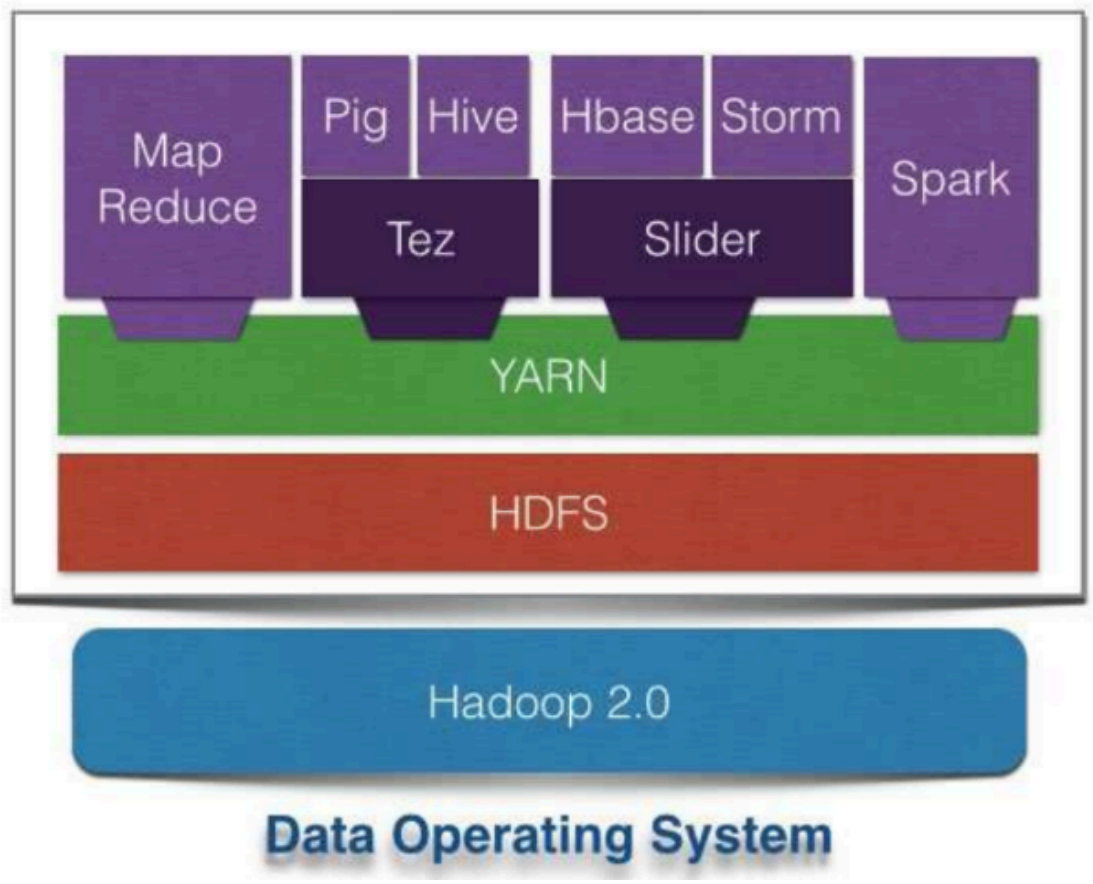
¿Qué es Hadoop?

Hadoop es un framework escrito en JAVA capaz de gestionar aplicaciones distribuidas, creado para abordar los desafíos asociados con el manejo de datos a gran escala.

Se caracteriza especialmente por:

- **Escalable:** Se pueden conectar nuevos nodos de forma sencilla.
- **Alta disponibilidad:** Los nodos pueden sustituirse de forma ágil.
- Flexible: permite configuraciones de todo tipo.

Esquema Hadoop



Los componentes de Haodoop son:

1. **Hadoop Distributed File System (HDFS):** Es un sistema de archivos distribuido diseñado para almacenar datos de manera distribuida en varios nodos de un clúster. HDFS divide los archivos en bloques y replica estos bloques en varios nodos para garantizar la tolerancia a fallos. Los bloques principales son:
    a. **Namenode:** Gestiona el acceso a los datos y el almacenamiento de metadatos.
    b. **Datanode:** Almacenamiento de datos.
    c. **BlockCaching:** Cacheo de bloques para permitir un acceso rápido.
    d. **Federación:** Escalado agregando name nodes.
2. **YARN (Yet Another Resource Negotiator):** Es un administrador de recursos que permite la gestión eficiente de recursos en un clúster Hadoop. YARN coordina el uso de recursos de manera que múltiples aplicaciones puedan

ejecutarse simultáneamente en el mismo clúster. Dentro del YARN se encuentra MapReduce.

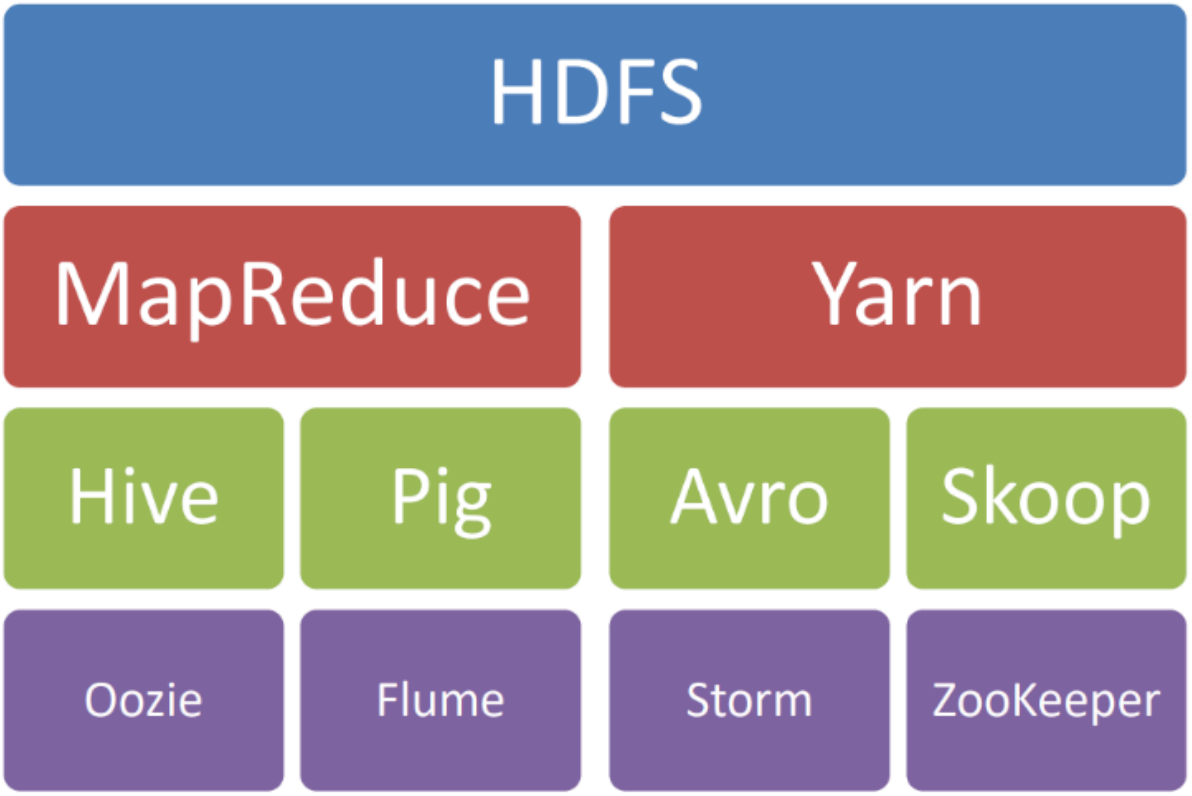
Ecosistema Hadoop

Una de las principales ventajas de Hadoop es el alto número de recursos de los que dispone. El conjunto de ellos se conoce como Ecosistema Hadoop.

Los más importantes son:

1. **Pig: E**s un lenguaje de alto nivel que traduce a “MapReduce”.
2. **Hive:** Convierte una transformación en lenguaje SQL en Pig o MapReduce.
3. **Ambari, Ganglia, Nagios:** Ofrecen una interfaz de acceso al clúster
4. **Sqoop:** Permite ejecutar aplicaciones MapReduce que introducen o extraen información de bases de datos SQL.
5. **Flume:** Sirve para introducir datos en streaming en Hadoop.
6. **Oozie:** Permite programar la ejecución de los Jobs de MapReduce.
7. **Storm:** Reproducción de los datos que no se han procesado correctamente la primera vez.
8. **Mahout:** Librería de Machine Learning que permite escribir aplicaciones MapReduce.

MapReduce es tan importante, que a menudo conceptualmente se considera un elemento independiente del YARN.



Conocimiento básico de HDFS

¿Qué es HDFS (Hadoop Distributed File System)?

Es un **sistema de archivos distribuido**, diseñado para almacenar y procesar **grandes volúmenes de datos** de forma eficiente y fiable. Está pensado para ejecutarse sobre **hardware estándar o de bajo coste.**

Una de las principales características de HDFS es su **alta tolerancia a fallos**. Los datos se almacenan de manera distribuida y replicada en varios nodos del sistema, de modo que la pérdida de uno de ellos no compromete la disponibilidad ni la integridad de la información.

HDFS está optimizado para aplicaciones que trabajan con **conjuntos de datos masivos.**

Arquitectura de HDFS: Sigue un modelo maestro-esclavo:

- **NameNode**: Es el nodo maestro del sistema. Gestiona el espacio de nombres del sistema de archivos y mantiene la información de metadatos, los permisos y la ubicación de los bloques que componen cada archivo.

- **DataNodes**: Son los nodos encargados de almacenar físicamente los datos. Cada DataNode gestiona los bloques que le asigna el NameNode.
- **Secondary NameNode**: Es un nodo auxiliar que almacena y consolida periódicamente la información de metadatos para facilitar la recuperación del sistema en caso de fallo.

Funcionamiento básico: Cuando un archivo se almacena en HDFS, se desarrolla del siguiente modo:

1. El **NameNode divide el archivo en bloques** y determina en qué DataNodes se almacenará cada uno.
2. El **NameNode gestiona las operaciones del sistema de archivos**, como abrir, cerrar o renombrar archivos y directorios.
3. Los **DataNodes crean, eliminan y replican bloques** siguiendo las instrucciones del NameNode.
4. La **ubicación de las réplicas** se optimiza para garantizar la fiabilidad de los datos, la alta disponibilidad y un uso eficiente del ancho de banda de red.
5. Los **datos del usuario nunca pasan por el NameNode**; este solo gestiona los metadatos, lo que evita cuellos de botella y mejora el rendimiento.

Los **DataNodes no tienen conocimiento del concepto de archivo** dentro de HDFS: únicamente almacenan bloques de datos, sin información sobre a qué archivo pertenecen.

Características clave de HDFS

- Los archivos en HDFS siguen un modelo de **una sola escritura.**
- Está optimizado para **lecturas secuenciales** de grandes volúmenes de datos.
- Prioriza la **consistencia, disponibilidad y tolerancia a fallos**, frente al acceso de baja latencia.

Conociendo Apache Spark

¿Qué es Apache Spark?

Es un framework de procesamiento de datos en tiempo real y por lotes de código abierto que proporciona un entorno rápido y unificado para el análisis

de datos y la **computación distribuida/paralela.**

Características clave

1. **Computación en memoria:** Permite procesar datos en memoria, lo que mejora el rendimiento en comparación con los sistemas que dependen del almacenamiento en disco.
2. **Agnóstico respecto a las fuentes de datos:** Puede trabajar con diversos tipos de datos, incluyendo datos estructurados, datos semiestructurados y datos no estructurados.
3. **Modelo de programación versátil:** Proporciona interfaces en varios lenguajes de programación (Scala, Java, Python y R). Además, tiene API para SQL, machine learning (MLlib), procesamiento de gráficos (GraphX) y streaming de datos (Spark Streaming).
4. **Compatibilidad con Hadoop:** Se integra bien con el almacenamiento distribuido Hadoop Distributed File System (HDFS) y puede utilizar Hadoop YARN para la administración de clústeres.

¿Por qué es útil Spark?

Es útil para lo que se usa principalmente:

- Transformación de datos
- Consultas de datos masivas
- Aplicación de streaming
- Machine Learning

¿Cómo funciona Spark?

Su funcionamiento es a modo "Maestro/Esclavo" donde:

- El proceso driver (Driver Node) gestiona la ejecución e informa al usuario.
- Los procesos ejecutores (Workers Nodes) procesan las tareas asignadas por el driver.

Los datos están repartidos en diferentes particiones y una tarea trabaja sobre ellas en paralelo.

Existen los **RDD ("*conjunto de datos resilientes"*).** Una vez creados no se pueden modificar, los cambios los transforman en otro RDD. También existen acciones para consultar valores.

Adentrándonos en el ecosistema Hadoop

- Common: Librería que permite realizar diferentes operaciones sobre el sistema.
- HDFS: Sistema de archivos distribuidos. Permite distribuir la información en nodos y, a través de replicación, asegurar la tolerancia a fallos.
- YARN: Gestor de recursos. Distribuye los trabajos entre diferentes nodos. Equilibra y balance los trabajos.
- Herramientas: Diferentes herramientas que utiliza el YARN (y por extensión el Hadoop sobre los datos del HDFS)
- MapReduce: Herramienta clave que me permite ejecutar operaciones bajo técnicas de **divide y vencerás.**

¿Qué es “Divide y Vencerás?

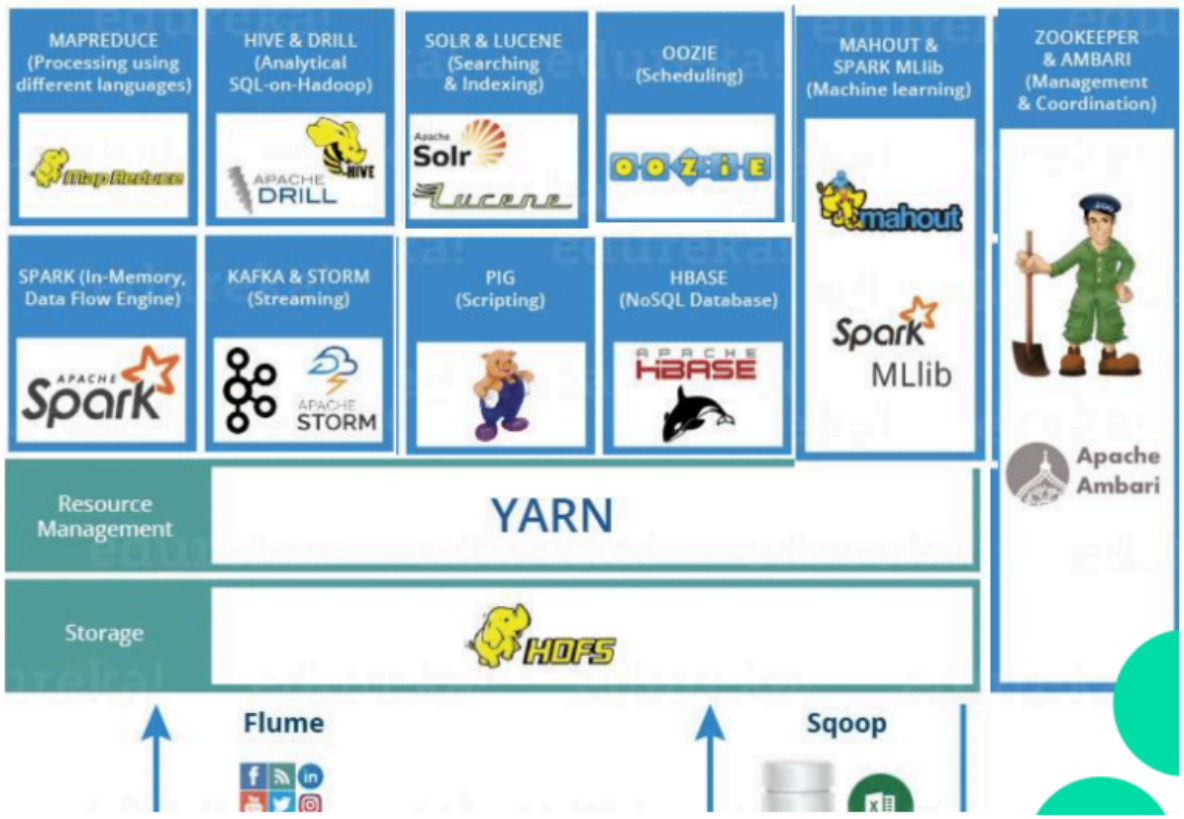
Es un enfoque de diseño de algoritmos y estrategia de resolución de problemas que consiste en dividir un problema grande en subproblemas más pequeños y manejables. Luego, se resuelven individualmente los subproblemas, y las soluciones se combinan para obtener la solución del problema original.

El proceso típico de "divide y vencerás" consta de tres pasos:

1. **Divide:** El problema se divide en subproblemas más pequeños y más fáciles de manejar.
2. **Vence:** Cada subproblema se resuelve de manera independiente.
3. **Combina:** Las soluciones de los subproblemas se combinan para obtener la solución del problema original.

Elementos del ecosistema Hadoop

El ecosistema se resume en la siguiente imagen:



Respecto a su arquitectura:

1. Trabajaremos con 2 capas:
    a. MapReduce: Resolución de problemas (tarea y trabajo)
    b. HDFS: Almacenaje (NameNode y DataNode)

MapReduce

Los datos se distribuyen en múltiples nodos dentro de un clúster. En Hadoop el programa se mueve hacia los datos, esto es clave para mejorar el rendimiento y reducir el tráfico de red

Dentro de la capa de procesamiento en arquitecturas Big Data existen dos grandes enfoques:

- Procesamiento por lotes (batch). → Hadoop
- Procesamiento en tiempo real (streaming).

¿Qué es MapReduce?

MapReduce es:

- Un modelo abstracto de programación.
- Una implementación concreta dentro del ecosistema Hadoop.
- Un módulo para procesar grandes volúmenes de datos de forma paralela y distribuida.

Se inspira en la técnica **“divide y vencerás”:**

- Dividir el problema en partes pequeñas.
- Procesarlas en paralelo.
- Unir los resultados parciales.

Una de sus grandes ventajas es que quita al desarrollador de:

- La gestión del hardware.
- La comunicación entre nodos.
- La tolerancia a fallos.
- La planificación de tareas.

El programador debe definir dos funciones: Map y Reduce

Funcionamiento general de MapReduce

MapReduce trabaja siempre con pares (clave, valor):

- Map
- Shuffle o Group by key (agrupación automática)
- Reduce

Características clave de MapReduce

Se pueden resumir en:

1. Procesamiento distribuido.
2. Paralelización automática.
3. Tolerancia a fallos.
4. Escalabilidad horizontal.
5. Basado en el modelo clave-valor.

6. Pensado para procesamiento por lotes (batch).
7. Además, el sistema es capaz de:
8. Reasignar tareas si un nodo falla.
9. Reejecutar procesos incompletos.
10. Aprovechar la localidad de los datos (data locality).

Arquitectura de MapReduce en Hadoop 1

En la primera versión de Hadoop (MapReduce 1), la arquitectura se basaba en dos componentes principales:

JobTracker (nodo maestro)

- Recibe el trabajo del cliente.
- Localiza los bloques de datos en HDFS.
- Asigna tareas a los nodos.
- Supervisa el progreso.
- Reasigna tareas en caso de fallo.

TaskTracker (nodos esclavos)

- Ejecutan tareas Map y Reduce.
- Informan continuamente al JobTracker.
- Ejecutan el código en el nodo donde están los datos.

Problema principal:

El JobTracker centralizaba demasiadas responsabilidades, lo que afectaba a la escalabilidad cuando el clúster crecía.

Evolución a Hadoop 2: YARN

En Hadoop 2 aparece YARN (Yet Another Resource Negotiator).

YARN separa:

- La gestión de recursos.
- El procesamiento de datos.

Nuevos componentes:

- **Resource Manager:** Gestiona los recursos globales del clúster (CPU, memoria, contenedores).
- **Node Manager:** Gestiona los recursos de cada nodo individual.
- **Application Master:** Se crea uno por cada trabajo. Controla la ejecución concreta de ese job.

Ventajas de YARN:

- Mayor escalabilidad.
- Mejor aprovechamiento de recursos.
- Permite ejecutar otros frameworks además de MapReduce (Spark, Tez, etc.).
- Mejor rendimiento global.

Con YARN, MapReduce deja de ser el único motor de procesamiento y pasa a ser uno más dentro del ecosistema.

Inconvenientes de MapReduce

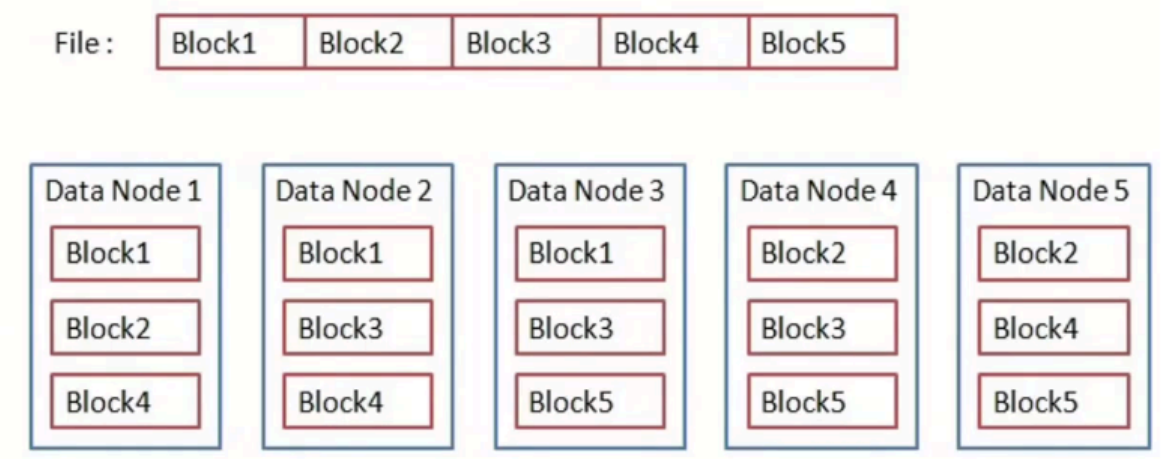
MapReduce presenta limitaciones:

- Escritura constante en disco de resultados intermedios.
- Alto tráfico de red en la fase Shuffle.
- Bajo rendimiento en algoritmos iterativos (machine learning).
- Código complejo y extenso (especialmente en Java).
- No es intuitivo para todos los tipos de problemas.

Comandos HDFS

El HDFS es el sistema de archivos distribuidos. Tiene el **NameNode**, donde está la información de guía de los nodos, identificado donde están los datos (los **DataNode**):

Para asegurar la tolerancia a fallos se utiliza la replicación. Aquí un ejemplo de archivo que usa la replicación en diferentes nodos:



Abrir un fichero en HDFS es realmente abrir información de diferentes nodos.

Para mantener la consistencia de los datos (tolerancia a fallos), hay operaciones útiles como:

1. **fsimage:** devuelve instantánea de espacio de nombres.
2. **editlog:** histórico de modificaciones.

📂 Comandos de HDFS

📂 Listar archivos y directorios

```
hdfs dfs -ls /ruta/del/directorio        # Lista archivos y directorios
hdfs dfs -ls -q /ruta/del/directorio     # Muestra los permisos en formato detallado
hdfs dfs -ls -R /ruta/del/directorio     # Listado recursivo
```

📂 Crear directorios

```
hdfs dfs -mkdir /(nombre que quiera)     # Crea un directorio llamado files en la raíz de HDFS
hdfs dfs -mkdir -p /ruta/del/directorio  # Crea directorios anidados si no existen
```

📂 Subir archivos a HDFS

```
hdfs dfs -put (archivo_local)    # Sube un archivo o directorio local a HDFS
hdfs dfs -put -f archivo_local /ruta/hdfs/   # Fuerza la subida, sobrescribiendo si existe
```

📂 Copiar archivos en HDFS

```
hdfs dfs -cp (si el archivo que necesitamos está en la raíz de HDFS: hdfs dfs -cp /*archivo que queramos* /files/ )
hdfs dfs -cp /ruta/origen /ruta/destino             # Copia un archivo o directorio dentro de HDFS
hdfs dfs -cp -f /ruta/origen /ruta/destino          # Sobrescribe si ya existe
hdfs dfs -copyFromLocal archivo_local /ruta/remota  # Copia un archivo local a HDFS
hdfs dfs -copyToLocal /ruta/remota archivo_local    # Copia un archivo desde HDFS al sistema local
```

📂 Mover archivos en HDFS

```
hdfs dfs -mv /ruta/origen /ruta/destino  # Mueve archivos o directorios dentro de HDFS
```

📂 Eliminar archivos o directorios

```
hdfs dfs -rm /ruta/del/archivo           # Borra un archivo
hdfs dfs -rm -r /ruta/del/directorio     # Borra un directorio y su contenido
hdfs dfs -rm -f /ruta/del/archivo        # Borra un archivo sin preguntar
hdfs dfs -rm -skipTrash /ruta/del/archivo  # Elimina directamente sin pasar por la papelera
```

📂 Ver contenido de archivos

```
hdfs dfs -cat (/el archivo que seleccionemos)   # Muestra el contenido de un archivo
hdfs dfs -tail /ruta/del/archivo  # Muestra las últimas líneas de un archivo
hdfs dfs -head /ruta/del/archivo  # Muestra las primeras líneas de un archivo
hdfs dfs -text /ruta/del/archivo  # Convierte archivos comprimidos a texto si es posible
```

Opciones avanzadas y útiles

```
hdfs dfs -ls                # Lista archivos y directorios
hdfs dfs -ls -q             # Muestra permisos y propietarios en un formato detallado
hdfs dfs -mkdir -p          # Crea directorios anidados si no existen
hdfs dfs -rm -r             # Borra un directorio de forma recursiva
hdfs dfs -rm -f             # Fuerza la eliminación sin confirmación
hdfs dfs -cp -f             # Fuerza la copia de archivos sobrescribiendo si existen
hdfs dfs -put -f            # Sobrescribe archivos al subirlos
hdfs dfs -text              # Intenta leer archivos comprimidos en texto plano
hdfs dfs -count             # Muestra el número de archivos y espacio usado en un directorio
hdfs dfs -du -s             # Muestra el tamaño total de un directorio
hdfs dfs -chown usuario:grupo /ruta  # Cambia el propietario de un archivo o directorio
hdfs dfs -chmod 755 /ruta   # Cambia los permisos de un archivo o directorio
hdfs dfs -touchz /ruta/archivo  # Crea un archivo vacío en HDFS
```